Midterm Review

LCD 151 · Methods in Computational Linguistics I

What will be covered on the midterm

1. Variables: naming, assignment, and updating values
2. Operators: arithmetic `+-/*` and value comparison `> >= < <= ==` etc
3. Calling functions: `print()`, `input()`, `len()`, `max()` etc
4. Calling methods: `.append()`, `.isnumeric()`, `.isalpha()`, `.lower()` etc
5. Data types: `str`, `list`, `int`, `float`, and `bool`
6. Strings and Lists: indexing and slicing
7. Conditionals: if, elif, and else
8. For loops: looping through lists (a small part)

On the exam day

1. You have 75 minutes to complete the exam.
2. If you finish early, you may submit your exam and leave after the first 30 minutes.
3. You may bring one 3 × 5-inch index card with handwritten notes on both sides.
4. No additional notes or electronic resources are permitted.

Exam Components (50pt)

Question Type	Points
True or False: 10 statements	10
Identify data values and types	5
Predict the output of code	10
Complete missing code and explain	10
Debugging * 2	15
Total	**50**

Review Session

Natural vs. Formal Languages

- **Natural language** develops through human use, such as English, Spanish, and signed languages.
- **Formal language** follows explicitly defined rules without ambiguity
  - Machine language: lowest-level language that a computer understands directly (sequences of binary digits 0s and 1s)
  - Programming language: higher-level language that humans can understand more easily, such as Python

In the first half of the semester, we've learned some of Python's basic rules and how to use them to write simple programs.

Variables

A variable is a **name** that refers to a **value** so one can use again in the future. To create a variable, we use `=` as an assignment statement.

```
n = 17  # Create a variable named n and assign it the value 17

new_list = [] # create an empty list

new_string = '' # create an empty string

x += 1  # Increase x by 1; equivalent to x = x + 1
```

Variable names

Variable names can be as long as you like. They can contain both letters and numbers, but they

1. can't begin with a number
2. can't contain special characters other than `_`
3. can't be python keywords such as `if`, `else`, `False`

If you give a variable an illegal name, you get a syntax error.

Data Type

In programming, every **value** has a **type** – or we sometimes say it “belongs to” a type.

- Integers `int`: whole numbers, such as 12 and -3
- Floats `float`: numbers with a decimal point, such as 12.0 and 3.5
- Strings `str`: text enclosed in quotation marks, such as "hello" and "12"
- Booleans `bool`: `True` or `False`
- Lists: sequences of items, such as `["cat", "dog"]`
- Dictionaries (dict): key–value pairs, such as `{"name": "Han"}` — coming soon!

Trace the Value and Type of `x`

Look at the following expressions, what are the **value** and **type** of `x` after each line?

Code	Value of `x`	Data type
`x = 5`	`5`	`int`
`x += 2`	`7`	`int`
`x = x / 2`	`3.5`	`float`
`x = str(x)`	`"3.5"`	`str`
`x = [x, "cat"]`	`["3.5", "cat"]`	`list`
`x = len(x)`	`2`	`int`
`x = x > 3`	`False`	`bool`

We will have one question similar to this in the exam.

Function

1. A named sequence of statements that performs some useful operation.
2. To **call** a function, write its name followed by parentheses.
3. The parentheses contain any **arguments** you pass to it.
4. Functions can take different numbers of arguments.

```
x = input()                  # No arguments
x = input("Enter a number")  # One argument: the prompt string

len("hello")            # One argument

max([3, 7])            # One argument: a list containing two items
max(3, 7)             # Two arguments: two integers

print("Hello", "Han", 5) # Multiple arguments
print(f"Hello {x}!")     # One argument - the entire f-string
```

Method

1. A method is a function associated with objects of a particular type.
2. To call a method, write the object that a method intends to work with, then a dot, and the method's name followed by parentheses.
3. Like functions, methods can take arguments inside the parentheses.

```
x = "Word"

x.lower()               # No arguments, return a new string

x.replace("d", "m")     # Two arguments, return a new string

x.isalpha()             # No argument, return a bool
```

Sample Exam Question

Look at this code, and answer the questions:

```
1   x = input("Enter a number: ")
2   if x.isnumeric():
3       print(f"The next number:{int(x)+1}")
```

1. Which line assigns a value to a variable?
2. Which lines call functions? Name the functions, and explain what they do
3. Which line calls a method? Name the method, and explain what they do.
4. What does the program print after the user enters 24?

Sample Exam Question Answer

Look at this code, and answer the questions:

```
1   x = input("Enter a number: ")
2   if x.isnumeric():
3       print(f"The next number: {int(x)+1}")
```

1. Line 1 assigns the string entered by the user to the variable `x`.
2. Lines 1 and 3 call functions:
   - Line 1: `input()`
   - Line 3: `int()` (convert `x` into an int) and `print()`
3. Line 2 calls the method `.isnumeric()`: returns True if **every character is a digit**
4. If user inputs 24, the output will be `The next number: 25`

Two Types of Sequences

String and List

String and List

Both string and list are **sequence**, which means it contains multiple values in a particular order. Indexing starts at 0.

String is a sequence of **characters**

Character	s	m	i	l	e
Index	0	1	2	3	4

List is a sequence of **objects** (could be mixture)

Objects	"cat"	"dog"	"elephants"
Index	0	1	2

`range()` also creates a sequence of **integers**. Eg. `range(5)` represents `0, 1, 2, 3, 4`

String and List: Similarity

Both are **sequences**, so we can use **indexing and slicing** with `[]`

Indexing

Indexing a string returns a **character**; indexing a list returns an **item**

```
word = "smile"
animals = ["cat", "dog", "bird"]

word[0]       # "s"
word[-1]       # "e" reverse indexing

animals[0]    # "cat"
```

String and List: Similarity

Slicing

Slicing a string returns a **substring**; slicing a list return a **sublist**

```
word = "smile"
animals = ["cat", "dog", "bird"]

word[1:4]     # "mil" --> substring!
animals[:2]  # ["cat", "dog"] --> sublists!
```

Note! An item is different from a list containing one item.

- `animals[0]` gives `"cat"` (type `str`)
- `animals[:1]` gives `["cat"]` (type `list`)

String and List: Difference

Strings are Immutable

1. You cannot change a character by assigning a new value to an index.

```
word = "Cat"
word[0] = "b"  # Error
```

2. String methods return a new string (original strings stay put)

```
word = "Cat"
word.lower()  # Returns "cat"
print(word)   # ?
```

How do we keep the updated string?

```
word = word.lower() # update the original "word" with the new one
print(word)   # Prints cat
```

String and List: Difference

Lists are mutable

1. You **can** change an item by assigning a new value to its index.

```
animals = ["cat", "dog", "fish"]
animals[0] = "bird"
print(animals) # ?
```

2. List methods change the original lists

```
animals = ["cat", "dog", "fish"]
animals.append("bird")
animals.remove("cat")
print(animals)    # ?
```

What if you do `animals.remove("cast")` or `animals = animals.append("bird")`?

A list inside a list

```
groups = [["sun", "moon", "star"],["cat", "dog"]]
```

What does each expression give?

```
groups[0]
groups[0][1]  # select an inner list first, then an item in that group.
groups[0][1:]
len(groups)
len(groups[1])
```

Boolean and Boolean expressions

1. A **Boolean** (`bool`) has one of two values: `True` or `False`. Note their first letter is capitalized.
2. A **Boolean expression** is an expression that its final evaluation is `True` or `False`.

```
score = 60
word = "cat"
submitted = False


# Boolean expressions

score >= 70              # Is score at least 70? NO, evaluate to False
"a" in word              # Does word contain "a"? YES, evaluate to True
submitted != False       # Is "submitted" not "False"? NO, evaluate to False
```

Sample Exam Question

Assume:

```
score = 65
word = "cat"
submitted = False
```

Predict `True` or `False` for each expression.

```
score > 60 and submitted
score > 60 or submitted
not submitted
"a" in word
len(word) != 3
```

In order: `False`, `True`, `True`, `True`, `False`.

Decision Structure

Use `if` (`elif`, `else`) to choose what codes to run under a certain condition

Two-way Decision

```
if condition_A:
    # do x if the condition is True
else:
    # do y if the condition is False
```

Multi-way Decision

```
if condition_A:
    # do x if condition_A is True
elif condition_B:
    # do y if condition_A is False and condition_B is True
else:
    # do z if neither condition is True
```

Sample Question: Debugging

A student wrote the following program to assign a Pass/Fail grade based on a number score. E.g. if `score = 85`, it will output `Strong pass` and then `Grading complete`.

The program contains **four** errors. Identify each error, explain the problem, and show how to correct it.

```
score = input("Enter your score: ")

if score >= 60:
    print("Pass")
if score >= 80:
    print("Strong pass")
else:
    print("Try again")
    print("Grading complete")
```

Sample Question: Debugging

```
score = input("Enter your score: ") # Error 1: Fail to convert the string to int

if score >= 60: # Error 2: wrong logic
  # should check 80 first so scores ≥80 receive “Strong pass.”
    print("Pass")

if score >= 80: # Error 3: wrong structure, use elif instead;
    # A separate if can cause to print both "Pass" and "Strong pass"
    print("Strong pass")

else:
    print("Try again")
    print("Grading complete") # Error 4: wrong indentation:
    # “Grading complete” should print for every score,
    # but currently prints only when the else block runs.
```

Loop Structure

Only For Loop will appear in the exam

A `for` loop visits each item

Use a for loop when you already have a sequence, such as a string or list, and want to perform an action on each item one by one **(definite loop).**

What does this program do? Explain each line.

```
numbers = [3, 8, 2, 10]
large_numbers = []
count = 0

for number in numbers:    # Question: what is "number"?
    if number >= 5:
        count += 1
        large_numbers.append(number)  # Question: what does append do?

print(large_numbers)
print(count)
```

Indentation Matters!

What is the difference between these programs?

```
# Program A
for letter in "cat":
    print(letter)
    print("Done")
```

```
# Program B
for letter in "cat":
    print(letter)
print("Done")
```

- A prints `Done` after every letter because it is inside the loop structure.
- B prints `Done` only once after all letters are printed.
- Indentation controls repetitions.

Classic question: A running tally

```
1  numbers = [3, 5, 2, 4]
2  total = 0
3  for number in numbers:
4      total = total + number    # Question: what's the shorthand for this?
5  print(total)                  # What would be the output?
```

Trace the value of `number` and `total`

Step	`number`	`total`
Before the loop	Not assigned yet	`0`
Round 1		
…		

1. What if I want to print `The current number is x` every time a new number comes in?
2. What if I want to print `The current total is x`?

Classic question 2: A running tally

(`break` will not appear in the midterm)

```
1  numbers = [3, 5, 2, 4]
2  total = 0
3  for number in numbers:
4      total = total + number
5      if total >= 10:
6        break                  # What does break do here?
7  print(total)                 # What would be the output?
```

1. Trace the value of `number` and `total`
2. What if I want to print `The total is now >= 10. Stop counting!`?

Debugging: Sales Bonus

A student wrote the following program. There are three bugs.

```
1  john_sales = [182, 258, 114, 124, 201, 270, 150, 319, 197, 220, 300, 98, 198]
2  weekly_bonuses = []
3
4  for sale in john_sales:
5
6    if sale >= 200:
7      weekly_bonus = sales + 50
8
9    else:
10     weekly_bonus = sale
11
12     weekly_bonuses.append("weekly_bonus")
13
14 print(weekly_bonuses)
15 print(sum(weekly_bonuses))
```

Debugging: Sales Bonus Answer

```
1   john_sales = [182, 258, 114, 124, 201, 270, 150, 319, 197, 220, 300, 98, 198]
2   weekly_bonuses = []
3
4   for sale in john_sales:
5
6     if sale >= 200:
7       weekly_bonus = sales + 50 # Error 1: typo, should be sale
8
9     else:
10      weekly_bonus = sale
11      weekly_bonuses.append("weekly_bonus")
12      # Error 2: Use the variable weekly_bonus, not the string "weekly_bonus".
13      # Error 3: This line is inside else, so only sales below 200 are added.
14        # Option 1: Align this line with if/else so it runs after either branch.
15        # Option 2: Also add weekly_bonuses.append(weekly_bonus) after Line 7
16  print(weekly_bonuses)
17  print(sum(weekly_bonuses))
```

Debugging: counting vowels

A student has the following code for "counting vowel" problem. The code contains **2** errors and has **2~3** places for optimization.

```
vowel = "aeiou"
consonants = "bcdfghjklmnopqrstwxyz"
vowel_count = 0
cons_count = 0

for char in text:
  letter = char.lower()

if letter in vowel:
  vowel_count = vowel_count + 1

elif letter in consonants:
  cons_counts = cons_count + 1
```

Review checklist

Some common mistakes from people's labs and homeworks:

- Use an undefined variable
- Missing `:`, `""`, `()`, `[]`
- Incorrect indexing or slicing:
  - indexing starts at 0;
  - slicing excludes the stop index.
  - slicing a string returns a substring; slicing a list returns a sublist. A list containing one item is different from the item itself: ["cat"] is a list, while "cat" is a string.
- Wrong indentation
- Unnecessary branching

Practice the topic where your prediction and Python's result disagree.

Good Luck 🍀